```
            raise UserNotFoundException()
          else:
            return User(Name(row[0]), PhoneNumber(row[1]),
row[2])
    except Error as e:
      raise e
```

In the above snippet, we are connecting to a database server named *mysqldb* using *root* as the user and *admin* as the password, in order to use the database schema named *glarimy*. It is fine to have such information in the code for an illustration, but surely not a suggested approach in production as it exposes sensitive information.

The logic of the *fetch()* operation is quite intuitive. It executes a SELECT query against the *ums_users* table. Recollect that the *AddService* was writing the user data into the same table. In case the SELECT query returns no records, the *fetch()* function throws *UserNotFoundException*. Otherwise, it constructs the *User* entity from the record and returns it to the caller. Nothing unusual.

The application layer

And finally, we need to build the application layer. The model is reproduced in Figure 2. It consists of just two classes: a controller and a DTO.

As we already know, a DTO is just a data container without any business logic. It is primarily used for carrying data between the *FindService* and the outside world. We just offer a way to convert the *UserRecord* into a dictionary for JSON marshalling in the REST layer:

```
class UserRecord:
  def toJSON(self):
    return {
      "name": self.name,
      "phone": self.phone,
      "since": self.since
    }
```

The job of a controller is to convert DTOs to domain objects for invoking the domain services and vice versa, as can be observed in the *find()* operation.

```
class UserController:

  def __init__(self):
    self._repo = UserRepositoryImpl()

  def find(self, name: str):
    try:
      user: User = self._repo.fetch(Name(name))
      record: UserRecord = UserRecord()
```

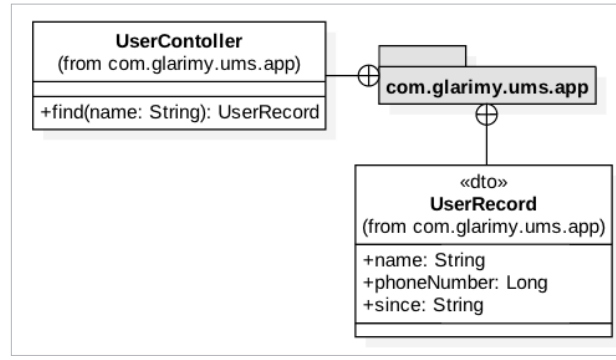

Figure 2: The application layer of *FindService*

```
      record.name = user.name.value
      record.phone = user.phone.value
      record.since = user.since
      return record
    except UserNotFoundException as e:
      return None
```

The *find()* operation receives a string as a name, converts it as a *Name* object and invokes the *UserRepository* to fetch the corresponding *User* object. If it is found, a *UserRecord* is created by using the retrieved *User* object. Recollect that it is necessary to convert the domain objects as DTO to hide the domain model from the external world.

The *UserController* need not have multiple instances. It can be a singleton as well. By overriding the *__new__* operation, it can be modelled as a singleton:

```
class UserController:
  def __new__(self):
    if not hasattr(self, 'instance'):
      self.instance = super().__new__(self)
      return self.instance

  def __init__(self):
    self._repo = UserRepositoryImpl()

  def find(self, name: str):
    try:
      user: User = self._repo.fetch(Name(name))
      record: UserRecord = UserRecord()
      record.name = user.name.getValue()
      record.phone = user.phone.getValue()
      record.since = user.since
      return record
    except UserNotFoundException as e:
      return None
```

We are done with implementing the model of the